Horquva OBA — Backend Foundation: Data Model & Roadmap

Organizational Brain & Agent (OBA) · Final Work Plan · Data Foundation → Phase 4 → Phase 5

For: Backend Team (Fizza & Shawal) · **Now:** build the data foundation · **Later:** Phase 4 & 5 integrations plug into the same foundation

The core idea (please read first): Build **one fixed "Organizational Data Model"** (a standard schema) that the whole product speaks. Right now, fill it with a rich, realistic dataset by hand. Later, in Phase 4 and 5, each tool you connect (GitHub, Slack, etc.) simply **maps its data into the same model** — the modules and the API never change. This one decision removes today's redundancy *and* makes the future integration work easy. Build the shape once; everything else plugs in.

The problem today: The system runs on one small file centred on a single person ("Robert"). The data is too thin and too uniform, so **every module produces almost the same result** — the 20-module product looks repetitive.

The goal now: Replace that file with a proper data model + a large, varied dataset, so that when all 20 modules run, **each one gives a clearly different, substantial result**. No live integrations or API fetching yet — that is Phase 4/5.

1. The Target Workflow (how the whole product should flow)

Sources (seed now · tools later)	**Normalization**	**Data Model (one schema)**	Access Layer (contract)	Modules + Engine

Build the middle three boxes now. The dataset feeds the left box today; real tools feed it in Phase 4/5 — **without changing anything to the right.**

2. What to Build Now

- **Define the Organizational Data Model** — the fixed list of entities, their fields, and how they relate (Section 3 below).
- **Stand it up as a real store** (proper tables/collections) — local or hosted is fine for now.
- **Seed it with a rich, varied dataset** (Section 5) — this is what kills the redundancy.
- **Expose one clean Data Access Layer** — a single, documented way the modules read data (this becomes the contract the AI team builds against).
- **Add validation & quality checks** — no duplicates, no missing required fields, basic logging.

3. The Organizational Data Model (canonical entities)

Entity	Key fields	Mainly feeds
Person	id, name, role, department, tenure, skills[], workload, manager	M01, M06, M13, M18
Agent / AI Tool	id, name, type, owner, cost, usage_count, adoption%, last_used	M07, M06, M13
Asset / System	id, name, type, owner, backup_owner, documented, criticality	M01, M03, M04, M05
Dependency (edge)	from, to, type (uses/feeds/reports-to), strength	M02, M03, M05, M16
Workflow + Steps	id, name, steps[{actor, action, duration}], frequency	M08, M16
Knowledge Asset	id, topic, holder, documented, criticality	M09, M10, M17
Event / Incident	id, date, type, entity, description, impact	M10, M11, M18

Decision	id, date, owner, factors[], outcome	M14, M10
Verification Record	item, status, verifier, date, expiry	M15
Governance / Policy	id, name, area, adherence, approval_status	M19
Accountability (RACI)	area, responsible, accountable, consulted, informed	M20
Snapshot (time-series)	date, metrics{headcount, workload, tool_cost, risk_index}	M11, M12, M17

> **Why this matters:** Every external tool in Phase 4/5 maps onto these same entities (e.g. GitHub → Person, Asset, Dependency, Event). Because the model is fixed, connectors are the *only* new work later — modules and API stay untouched.

4. Build a Rich, Varied Dataset (kills the redundancy)

- **30–50 people** across departments, with varied roles, tenure, skills, and workload — not all on one person.
- **Spread ownership & coverage** — different owners; some with backups, some without; some documented, some not. Variety is the point.
- **10–15 tools/agents** with real usage, cost, adoption, last-used.
- **8–12 workflows** with steps, actors, frequency, and timing.
- **A varied dependency graph** — chains, busy hubs, and isolated nodes.
- **4–6 monthly snapshots** so trends exist (needed by Predictive, Forecasting, Memory, Learning).
- **Activity logs** — incidents, decisions, verifications, governance records, accountability links.
- Keep it one believable company scenario, but deep.

5. Exactly What Each Module Needs (M01–M20)

Module	What it should reveal	Data it needs
M01 Ownership	Who owns what; over-concentration	Owner per asset; per-person asset counts
M02 Dependency	How things depend on each other	Dependency edges + type + strength
M03 Risk / SPOF	Critical items with no backup	Backup yes/no, dependents count,

	that many rely on	criticality
M04 Recommendation	Concrete fixes to suggest	Per-asset issue flags (no doc / no backup / overload)
M05 What-If	“If X leaves, what breaks?”	Dependency graph + owner + criticality
M06 Human-Agent Map	Who works with which agents/tools	Human ↔ agent assignments + frequency
M07 AI Tool	Used, costly, or idle tools	Usage, cost, owner, adoption, last-used
M08 Workflow	How work flows; where it jams	Workflow steps, actors, frequency, time
M09 Knowledge Risk	Undocumented critical knowledge	Knowledge assets + doc status + holder
M10 Memory	What’s at risk of being forgotten	Event log + decisions + tenure
M11 Predictive Risk	Rising risks	Time snapshots + leading indicators
M12 Forecasting	Where numbers are heading	Time-series of key metrics
M13 Human-AI Collab	Human vs AI balance + handoffs	Task assignments + handoff records
M14 Decision	Decision quality & ownership	Decisions log: owner, factors, outcome
M15 Verification	What needs checking + status	Verification records + status + expiry
M16 Orchestration	Coordination of agents/workflows	Orchestration tasks + assignments + deps
M17 Learning	Knowledge capture & reuse	Capture events over time + follow-through
M18 Continuity	Coverage if key people lost	Backup + succession + role criticality
M19 Governance	Policy & compliance posture	Policy records + adherence/approval

M20 Accountability	Who is responsible for what	RACI links + accountability gaps

6. How This Makes Phase 4 & 5 Easy (the payoff)

Phase 4 — Connect Core Tools: GitHub, Slack, Jira, Linear. Each connector just reads the tool and **maps it into the existing data model** (e.g. GitHub commits → Person activity + Asset ownership + Events). No module or API change needed.

Phase 5 — Enterprise Scale: Google Workspace, Microsoft 365, Salesforce, Calendars, Enterprise DBs — same pattern: map into the same model. The work becomes "write one mapper per tool", which is fast and low-risk.

> **In short:** because the data model and access layer are fixed now, every future integration is a small, isolated piece of work — not a rebuild. That is what makes the project industrial-grade and the workflow smooth.

7. Definition of Done (the quality bar)

- One documented Organizational Data Model + a clean Data Access Layer the AI team can build against.
- A large, varied dataset loaded — **no two modules produce the same result**.
- Time-based modules show real trends (because history/snapshots exist).
- Validation + logging in place; data is clean and consistent.
- A short note showing how a sample tool (e.g. GitHub) would map into the model — proving Phase 4 readiness.

Bottom line: The modules are fine — data is the bottleneck. Backend's job now is to build **one fixed data model + a rich, varied dataset + a clean access layer**. This removes the redundancy immediately

(every module lights up differently), and because the shape is fixed, Phase 4 and Phase 5 integrations later

become simple “map the tool into the model” tasks — easy for backend, easy for the AI team, and a

workflow that holds up at real industrial scale.